X Algorithm Reach Report

Repo-based analysis of how the For You recommendation system retrieves, filters, scores, and blends posts - plus practical reach guidance for X.

6	19	14+	P(action)
Main pipeline stages	Phoenix action heads	Core post filters	Core ranking idea

Important scope
This report is based on the code in this repository. The Phoenix README describes the release as a mini/frozen representative model; production is larger and continuously trained. Several feature-switch parameter values are referenced but not included in the checked-in files, so exact live numeric weights are unavailable.

Executive Summary

The repo implements a personalized predicted-engagement system. It gathers viewer context, retrieves candidate posts from in-network and out-of-network sources, removes ineligible or risky candidates, predicts action probabilities with Phoenix, combines positive and negative action predictions into a rank score, then selects and blends the final For You feed.

The practical takeaway: reach is not one universal score. A post must first enter retrieval, pass filters, score well for a specific viewer, avoid negative feedback predictions, and survive diversity, visibility, and feed-blending constraints.

What The Algorithm Favors

Signal	Evidence in code	Creator move
Positive action probability	Favorites, replies, reposts, shares, profile clicks, clicks, VQV, dwell, quote clicks, follow-author.	Create posts that make people respond, share, read/watch, or follow.
Topical match	Phoenix retrieval uses user history embeddings; topic sources use followed/new-user topic IDs.	Own a repeatable niche so the model learns who should see you.
Freshness	Thunder keeps recent posts; AgeFilter removes old candidates; model embeds post age buckets.	Post when the target audience is active and maintain cadence.
Social proximity	In-network source is explicit; author/social graph metadata are hydrated.	Build real followers in the niche; their early actions seed later OON retrieval.
Viewer retention	Dwell and continuous dwell-time features are modeled; video quality view has explicit scoring paths.	Use hooks, structure, media, and clear payoff to increase engaged time.
Diversity and originality	Repeated authors are attenuated; duplicate/conversation filters collapse repeats.	Do not flood. Make each post distinct enough to deserve a slot.

How The Algorithm Behaves

Stage	What the repo does	Reach implication
Viewer context	Hydrates action history, follows, mutes, blocks, topics, demographics, IP/device, seen and served history.	Personalization starts before any post is scored.
Candidate retrieval	Pulls in-network posts from Thunder and out-of-network posts from Phoenix, Phoenix Topics/MoE, TweetMixer, and cache.	A post must enter a candidate pool before ranking can help it.
Candidate hydration	Adds author/post metadata, quote data, video duration, media, subscription, language, topic labels, and social graph signals.	Missing or risky metadata can remove or weaken a candidate.
Hard filters	Drops duplicates, old posts, self posts, inaccessible subscription posts, seen/served posts, muted keywords, blocked/muted authors, excluded topics, and disallowed videos.	Many reach losses happen before scoring.
Ranking	Phoenix predicts engagement probabilities; RankingScorer combines positive and negative action predictions; optional VMRanker can rerank.	The score is predicted future value for this viewer, not raw global likes.
Selection and blending	Top K by score, then visibility filtering, conversation dedup, ads/prompts/who-to-follow/push-to-home blending.	Even high scoring content can be trimmed for safety, diversity, or product modules.

What Hurts Reach

Risk	Evidence in code	Do not do this
Negative feedback	Predicted not-interested, block, mute, report, and not-dwelled scores feed into ranking with negative weights.	Avoid rage bait, misleading hooks, low-trust claims, and report-prone content.
Spam or low-quality replies	Grox has reply spam detection and reply ranking plans.	Do not mass-reply, copy/paste, tag bait, or hijack unrelated large accounts.
Safety or policy risk	Visibility filtering and safety labels include NSFW, gore/violence, do-not-amplify, adult, hate/abuse, illegal/regulated, violent speech, self-harm.	Keep content policy-clean if reach is the objective.
Repetition and duplicates	Duplicate post IDs, retweets of same content, previously seen/served IDs, and conversation duplicates are removed.	Avoid reposting the same idea repeatedly or making near-identical replies.
Poor topic fit	Topic filters remove excluded topics and can require matching new-user/followed topics.	Do not chase every trend if it confuses your audience/content embedding.
Access and relationship blockers	Muted keywords, blocked/muted authors, author-blocks-viewer, protected/ineligible subscription content are filtered.	Avoid terms your audience commonly mutes and keep distribution public where possible.

Actionable Reach Playbook

Goal	What to do
Enter retrieval	Build a clear niche, use consistent language/topics, and get early engagement from followers who resemble the audience you want X to find.
Win ranking	Optimize for replies, reposts, dwell, profile clicks, follows, and shares while minimizing not-interested, mute, block, report, and instant-skip behavior.
Survive filters	Stay fresh, original, public, policy-safe, non-duplicate, non-spammy, and aligned with viewer topics and muted-keyword constraints.

Specific Posting Guidance

- Pick a narrow audience and stay semantically consistent. The retrieval model is embedding-based, so clarity of niche helps the system infer who should see you.
- Design posts for multiple positive actions: reply, repost, share, profile click, follow, dwell, click, and video quality view. Do not optimize only for likes.
- Use a strong first line, clear structure, and a payoff that rewards reading or watching. Dwell and continuous dwell-time are modeled.
- Create original posts rather than many near-duplicates. Duplicate, retweet, seen/served, and conversation dedup filters limit repeated exposure.
- Build early engagement from relevant followers. In-network candidates avoid the out-of-network score multiplier and can seed later discovery.
- Treat negative feedback as expensive. Anything that predicts not-interested, mute, block, report, or not-dwelled works against distribution.
- Stay policy-safe and advertiser-safe. Visibility filtering and safety labels can remove or restrict posts after ranking.

Content Format Notes

- Text: Lead with a clear hook, state the value quickly, and avoid vague bait.
- Threads: Use only when the idea needs it; repeated branches from one conversation can be deduped.
- Video: Quality views matter only when eligible video duration paths apply, so use real watchable value rather than filler clips.
- Replies: High-signal replies to relevant conversations can help, but spammy replies are explicitly analyzed in Grox paths.
- Topics: Consistency helps retrieval. Trend-chasing outside your niche can dilute the audience model.

Do Not Do These

- Do not mass-reply, tag-bait, copy-paste, or hijack unrelated large accounts.
- Do not post repeated versions of the same content in a burst.
- Do not create content likely to be muted, blocked, reported, or marked not interested by the audience you want.
- Do not rely on NSFW, gore, violent, hateful, illegal/regulated, or other safety-risk material for reach.
- Do not overfit to raw likes. The ranker considers many action probabilities, including negative outcomes.
- Do not ignore freshness. Old posts can fall out through age filters and Thunder retention.
- Do not make subscriber-only content your main reach vehicle if the goal is broad non-subscriber distribution.

Key Source Areas Reviewed

Area	Representative files
Home mixer	home-mixer/for_you_server.rs; home-mixer/candidate_pipeline/phoenix_candidate_pipeline.rs
Pipeline framework	candidate-pipeline/candidate_pipeline.rs
Ranking formula	home-mixer/scorers/ranking_scorer.rs; phoenix/runners.py
Retrieval model	phoenix/recsys_retrieval_model.py; phoenix/run_pipeline.py
Candidate isolation	phoenix/grok.py; phoenix/recsys_model.py
Filters	home-mixer/filters/*.rs
Safety/content classifiers	grox/plans/*.py; grox/classifiers/content/*.py

Caveats

- This is a partial OSS repository, not a complete guarantee of live X production ranking behavior.
- Runtime feature-switch values and some external crates are not present, so exact numeric weights and thresholds cannot be extracted from this checkout.
- No SimClusters, RealGraph, TwHIN, or direct verified-status scoring hooks were found in the reviewed files.
- Several paths are environmental or optional, including VMRanker, cluster selection, ad blending, and feature-switch controlled behavior.

Generated on 2026-05-15 23:03 from repository analysis of /Users/shreyaspandey/Coding_Files/oss/x-algorithm.